CS 4510 Automata and Complexity 1/17/2024

Homework 2: NFAs and Regular Language Properties

Devin Fromond *Due:1/27/2024*

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments. Three bonus points if you use LaTeX, and our template. You may collaborate with other students, but any written work should be your own.

1. (a) Let L be any language. Define $L' = \{w \in \Sigma^* \mid wx \in L, x \in \Sigma^*\}$. Prove that if L is regular, then so is L'. This language is essentially the language of prefixes of L.

   Answer: We begin by building a DFA such that $M' = (Q, \Sigma, \delta, q_0, F')$ that accepts the language L'. We begin by defining F' such that

   $$F' = \{q \in Q | \textit{there exists a string } x \in \Sigma^* \textit{ such that } \delta(q, x) \in F\}$$

   In layman's terms, we defined a new F (F') that is the set of all the states in Q such that Q connects to F. Since Q is finite, we easily observe F' is found by starting from the states in F and working backward.

   We observe this to be true by the following simple proofs of directionality:

   - If $w \in L'$, there is x such that $wx \in L$. Therefore, we find $\delta(q_0, w) \in F'$.
   - Similarly, we find if $\delta(q_0, w) \in F'$, there exists an x such that $\delta(q_0, wx) \in F$, or $wx \in L$.

   Therefore, we easily find $L(M') = L'$. Since M' is a DFA, L' is regular.

   (b) The symmetric difference of A and B, denoted $A \oplus B$, is the set of elements in A or in B, but not in both. For example, if $L = \{\varepsilon, 0\}$ and $L' = \{\varepsilon, 1\}$ then $L \oplus L' = \{0, 1\}$. Prove that REG (the set of regular languages) is closed under symmetric difference.

   Answer: By identity, we have that for all languages A, B,

   $$A \oplus B \;=\; (A \cup B) \;\cap\; (A \cap B)^c.$$

   Since regular languages are closed under union, intersection, and complement, the right hand side is a regular language whenever A and B are regular. Therefore, we find $A \oplus B$ to be regular.

2. Let $\Sigma = \{a, b\}$. Let L be a regular language. Suppose that the DFA for L has the property that $|F| = 1$. Prove that if $x, y \in L$ then for all $z \in \Sigma^*$ both $xz, yz \in L$ or both $xz, yz \notin L$.

   Answer: We define a DFA M to be $M = (Q, \Sigma, \delta, q_0, F)$ that accepts L whenever $|F| = 1$. For ease of reading, we define this accepting state to be q_{acc}. Since $x, y \in L$, when M processes x starting at q_0, it must end in q_{acc}. Similarly, we observe processing y from q_0 also ends in q_{acc}.

   We now consider strings such that $z \in \Sigma^*$. Since $\delta(q_0, x) = q_{\text{acc}}$ and $\delta(q_0, y) = q_{\text{acc}}$, reading z from q_{acc} yields a single next state (by the DFA's definition). Therefore, we find the following:

$$\delta(\, q_0,\, xz\,) = \delta\big(\, \delta(q_0, x), z\big) = \delta(q_{\text{acc}}, z)$$

   and

$$\delta(\, q_0,\, yz\,) = \delta\big(\, \delta(q_0, y), z\big) = \delta(q_{\text{acc}}, z).$$

   By observation, we find both xz and yz end in the *same* state. Therefore, we find either both xz and yz are accepted (if that state is q_{acc}) or both are rejected (if that state is not q_{acc}).

   Finally, we observe that when $x, y \in L$, for all $z \in \Sigma^*$ we have $xz \in L$ only if $yz \in L$.

3. Let $\Sigma = \{a, b\}$ and consider the language $(a \cup b)^* a (a \cup b)^{n-1}$.

   (a) Characterize what strings are in this language, in english

   Answer: We observe the language includes all strings whose length $\geq n$ over $\{a, b\}$ such that the n-th symbol from the right is an a.

   In Layman's terms, if we count backwards from the last character, the n-th character must be a.

   (b) Describe an NFA which decides/accepts this language in $n + 1$ states.

   Answer: We can define an NFA with $n + 1$ states in the subsequent steps. We begin by labeling the states

$$0, 1, 2, \ldots, n,$$

   with 0 being the starting state and n being the only accepting state. Next, we define the transitions to be:

   - From state 0, when processing a or b, we remain in state 0. In Laymans terms, this represents reading characters before we select which one is the n-th from last character.
   - From state 0, when processing an a, we observe a nondeterministic transition to state 1. This signifies that this a is the one that should be n-th from the end.
   - From each state i (for $1 \leq i < n$), when processing a or b, we move to the state $i + 1$.
   - Finally, we find state n is the unique accept state.

(c) Describe a DFA which decides/accepts this language in 2^n states.

Answer: Ultimately, a DFA must *remember* sufficient information to decide whether the n-th symbol from the right was a. Therefore, we must track the last $n-1$ symbols that are processed (this value may be fewer if the input is not of sufficient length).

We begin by defininining the states of the DFA as *all possible distinct strings* of length at most $n-1$ over $\{a, b\}$. We easily find there exist $\sum_{k=0}^{n-1} 2^k$ such strings, which (through simplification) is equivalent to $2^n - 1$ if we exclude a distinct dead configuration.

However, we observe we find exactly 2^n resulting states (counting a dead state explicitly) as a result of the way we store this extra piece of data.
In simpler terms:

- Each state records the suffix (up to length $n-1$) of the processed input.
- After processing a new symbol, the DFA updates the stored suffix (shifting left, dropping the oldest character if needed, and appending the new one).
- We define the accepting states as the states that are exactly those whose stored suffix has length $n-1$ and whose first character is a (In Layman's terms, if we saw at least n symbols, the n-th from last was a).

Therefore, we find counting the number of distinct ways to store up to $(n-1)$-length suffixes plus a dead or incomplete marker leads to 2^n states.

(d) Prove there is no DFA of any fewer states.

Answer: We begin by considering all 2^n strings of length n over $\{a, b\}$. We find these strings to be of length-n and, in addition, must reach a *distinct* state in any DFA for the language

$$(a \cup b)^* \, a \, (a \cup b)^{n-1}.$$

We explore the rationality below:

- We find a string of length n is accepted only when its first character is a.
- Now, we consider two different strings $x \neq y$ of length n that are mapped by the DFA to the same state. Exploring this, we find the following:
  - If one of them, for argument sake defined as x, has first character a and is therefore in the language, while the other, y, does not, then upon reading the strings fully, the DFA would either accept both or reject both (since they end in the same state). This fact contradicts the fact that one should be accepted and the other rejected.

Therefore, we observe no two distinct length-n strings can end in the same DFA state. Since there are 2^n such length-n strings, the DFA requires a minimum of 2^n distinct states not have a contradiction.
Therefore, we find that for any DFA such that $(a \cup b)^* \, a \, (a \cup b)^{n-1}$ must have at least 2^n states.

4. (a) Consider the power set construction. If a state is unreachable in the output DFA, what does this intuitively mean in the original NFA for the corresponding set of states?

Answer: We observe if a subset of NFA-states is unreachable in the DFA produced by the power-set construction, then there is no valid string input in the original NFA whose set of possible current states corresponds to that subset. As a sanity check, we find this subset never occurs as the active set of states when reading from the NFA's start state.

(b) Let $N = (Q, \Sigma, q_0, \delta, F)$ be an NFA with the following properties:
1. For each $q \in Q$ and $a \in \Sigma$, $|\delta(q, a)| \leq 1$.
2. There are no epsilon transitions.

Suppose you constructed a DFA from N using the power set construction. Futhermore, suppose you removed all states not reachable from the start state by any string. Prove that this DFA has at most $|Q| + 1$ states.

Answer: We observe that since $|\delta(q, a)| \leq 1$ for each q and a, from any single NFA state q and any letter a, there is at most one possible next state. In other words, there is no branching on a single symbol.

We find in the subset construction, the start state of the DFA is $\{q_0\}$. When the DFA processes a letter a, we look at all states in the current subset S and gather the (no more than one) next state each can go to on input a. However, we find that since from a single state q there is at most one transition on a, from a set S of states, the next subset must have a size of $\leq |S|$. Additionally, we find that with no ε-transitions, we never add extra states. Starting in $\{q_0\}$, each move on a letter yields either a single next state or the empty set (for no valid transition).

Therefore, we find every reachable subset of states in the DFA must be either a single state $\{q\}$ for some $q \in Q$ or the empty set $\varnothing$. Therefore, when considering the DFA's reachable states, there can be at most one empty set in addition to one singleton set for each state $q \in Q$. In total:

$$1 \text{ (for } \varnothing) \ + \ |Q| \text{ (for all singletons)} \ = \ |Q| + 1.$$

Finally, we find that after removing unreachable subsets, the resulting DFA has at most $|Q| + 1$ states.